RISK ASSESSMENT · NEXT CHAPTER

thunk — Software Project Risk Assessment

Project / System: thunk (offline systems course for the reentry population)

Assessor: Penn Porterfield · **Date:** 2026

D / V / F = the dimension the risk threatens: `D` Desirability `V` Viability `F` Feasibility. **L** = likelihood, **C** = consequence, each 1–5. **Risk = L × C** (max 25): `15–25 high` `8–14 medium` `1–7 low`. Ordered by initial risk.

RISK	HOW COULD IT HURT?	DVF	INITIAL	EXISTING CONTROLS	ADDITIONAL MITIGATION	FINAL
Curriculum scope unclear or underestimated	Scope creep; the Aug 2026 demo slips; solo-builder burnout	D	16 4 × 4	Written spec and phased plan with explicit non-goals; Phase 1 already built and passing 24 tests; milestones tracked in-repo	Freeze the demo to Phase 1; defer the web GUI, DOOM sim, and the M6-M7 open-source track; validate estimates against the built slice	6 2 × 3
Facility review blocks the software inside	The primary channel (inside the wall) closes; the core audience is unreachable	V	15 3 × 5	Offline by design: one static-musl binary, no network, no crypto, assets embedded at compile time; vocab-lint CI gate; targets the walled-garden laptops facilities already deploy	Early review by a DOC/facility contact; a plain-language security and review packet with a threat model; a single-facility pilot before scaling	8 2 × 4
The DOOM finale reads as a game and trips facility review	A playable game is treated as entertainment or contraband and the course is rejected on that basis	V	15 3 × 5	The finale is framed and presented as an engineering demonstration - a real program on a display driven from the bare metal up, not entertainment	A non-game bootable demo (graphics/test pattern) as an alternate finale for strict facilities; make the finale configurable; document the framing in the review packet	8 2 × 4
Weak or misjudged labor-market demand (market realism)	Graduates cannot convert systems and Rust skills into work; the core promise fails	V	12 3 × 4	A verified underserved niche; portable public artifacts that signal skill across many roles, not only kernel jobs (83% of orgs weigh open-source contributions)	Position outcomes for a range of roles, not only kernel work; set honest expectations - differentiation, not a guaranteed kernel job; track placement in a pilot	6 2 × 3

RISK	HOW COULD IT HURT?	DVF	INITIAL	EXISTING CONTROLS	ADDITIONAL MITIGATION	FINAL
Simulator cannot convincingly reach the DOOM finale	The payoff and the core differentiator weaken; costly late rework	F	12 3 × 4	First simulator slice built and tested (bus + panel + boot splash); an SpiBus trait gives the sim-or-real seam; doomgeneric is proven-portable	Build a DOOM-on-sim-panel vertical slice early to de-risk it; keep a simpler bootable demo as a fallback finale	6 2 × 3
Insufficient testing or inaccurate content	Learners are taught something wrong; credibility and trust are lost; support burden rises	F	12 3 × 4	24 passing tests; a self-validating content loader (asserts every check's answer grades correct); CI runs fmt, clippy (deny warnings), tests, and vocab-lint	Mentor/expert review of every lesson before release; grow coverage on the simulator and content; user-acceptance testing with a pilot learner	6 2 × 3
Learners get discouraged and drop out (retention)	Low completion undercuts outcomes and the whole case for the program	D	12 4 × 3	Self-paced and mastery-based, no failing grade; cannot get stuck (help always on, no dead ends); early visible payoff; tracked progress	A small peer cohort plus a mentor who has walked the path; tiny early wins before hard material; measure completion in a pilot	6 2 × 3
Built but not adopted - no facility or org deploys it	The course clears review and is free, yet never reaches learners; the effort is wasted	D	12 3 × 4	Next Chapter as first cohort and champion; free with zero infrastructure lowers adoption cost; complements, not competes with, existing programs	Land one lighthouse facility pilot and publish outcomes; partner with an org that already has facility delivery rails; onboarding is a single copied binary	8 2 × 4
Privacy breach of a vulnerable population's data	Exposed progress, answers, or typed input could harm a learner's standing inside or on release	D	10 2 × 5	Offline and local-only: no accounts, no network, no telemetry, no tracking; minimal data (a local progress file); nothing leaves the machine	Per-user state isolation and an easy wipe on shared machines; store no more free text than a check needs; document data handling; least-privilege file access	4 1 × 4
Open-source first-patch step overwhelms beginners	Discouragement at the most important moment; wasted maintainer time and reputational cost	D	9 3 × 3	Scope kept small (docs, staging fixes); established on-ramps (kernelnewbies, LKMP, Outreachy); meritocracy framing separates code critique from the person	Internal review of every patch before it is sent; a scaffolded first-issue list; a buddy for each first submission	4 2 × 2
No sponsor or funding to sustain / equip open-build learners	Hardware out of reach for outside learners; long-term sustainability at risk	V	9 3 × 3	Free and open source; simulator-first means a zero-hardware floor; the $200 Saleae writeup bounty is collectable now	Apply for education and reentry grants; Saleae and hardware-partner outreach; keep every outcome reachable with no hardware at all	4 2 × 2

RISK	HOW COULD IT HURT?	DVF	INITIAL	EXISTING CONTROLS	ADDITIONAL MITIGATION	FINAL
Curriculum drifts out of date as the kernel and Rust evolve	Version-specific claims age; content loses accuracy and credibility	F	9 3 × 3	Time-sensitive claims are date-stamped and sourced; content is plain markdown, cheap to revise; the core concepts (syscalls, mmap, SPI) are stable	A review cadence on version-specific claims; keep current-state facts in one place, separate from durable concepts; re-verify each cycle	4 2 × 2
The market gap closes or the differentiator is overtaken	An existing program adds systems, or the Rust SPI abstraction lands upstream, eroding the novelty claim	V	9 3 × 3	The differentiator is verified and date-stamped; the durable value (a portfolio and a public record) survives a crowded niche; first-mover plus lived experience is defensible	Re-verify the landscape each cycle; lead with durable value, not novelty; if Rust SPI lands upstream, pivot the capstone to the next unfilled area (e.g. GPIO)	6 2 × 3
Dependency on external communities and the host org	Maintainers reject the cohort's patches, or a partner deprioritizes; the payoff or reach weakens	V	9 3 × 3	Start with low-bar, high-acceptance contributions; multiple on-ramps, not one; mentor pre-review raises acceptance odds; the course stands alone without any single partner	Cultivate a friendly-maintainer and first-issue list; diversify host and partner relationships; keep the curriculum partner-independent	4 2 × 2
Solo maintainer unavailable (key-person risk)	Knowledge loss; the project stalls; the lived-experience voice is lost	F	8 2 × 4	Specs, plans, curriculum, and code are open source and documented in-repo (PRD, design spec, phase plan, README); CI encodes the quality bar	Grow co-maintainers out of the community; write an onboarding and contributor guide; identify a backup content reviewer	6 2 × 3

Reading the residuals. Three risks are deliberately left at `medium` after mitigation — **facility review**, the **game-perception of the DOOM finale**, and **adoption** — because each depends on a decision partly outside the team's control (a reviewer's call, an organization's choice to deploy). They are the active watch-list. The rest fall to low once mitigations are in place, largely by construction: an offline, no-network, no-hardware, review-safe design keeps most of these risks low to begin with. The highest-consequence items (a rejected build, a privacy exposure) are met with the strongest controls, which is where the effort belongs.